



An Intelligent Metaheuristic Algorithm for Optimal Tuning of Control System Parameters

Abbas Moloody^{1*}, Azizan As'array¹, Tang Saihong¹, Raja Kamil²

¹ Department of Mechanical and Manufacturing Engineering, Faculty of Engineering, Universiti Putra Malaysia (UPM), 43400 Serdang, Malaysia

² Department of Electrical and Electronics Engineering, Faculty of Engineering, Universiti Putra Malaysia (UPM), 43400 Serdang, Malaysia

* Correspondence: Abbas Moloody (a.moloody@gmail.com)

Received: 07-02-2026

Revised: 08-11-2026

Accepted: 08-21-2026

Citation: A. Moloody, A. As'array, T. Saihong, and R. Kamil, "An intelligent metaheuristic algorithm for optimal tuning of control system parameters," *J. Complex Multiphys. Eng. Syst.*, vol. 1, no. 4, pp. 359–370, 2026. <https://doi.org/10.56578/jcmes010403>.



© 2026 by the author(s). Published by Acadlore Publishing Services Limited, Hong Kong. This article is available for free download and can be reused and cited, provided that the original published version is credited, under the CC BY 4.0 license.

Abstract: This paper presents a unique Modified Differential Evolution Optimization Algorithm (MDEOA) for the intelligent modification of proportional–integral–derivative (PID) controller settings. The proposed MDEOA is developed with particular emphasis on future PID controller tuning applications. Benchmark evaluations demonstrate significant improvements in convergence speed and optimization quality compared with Classical Differential Evolution (CDE), indicating its potential suitability for intelligent control parameter optimization. The study tackles the enduring drawbacks of CDE in control parameter optimization, including its inadequate exploration-exploitation balance and sluggish convergence when adjusting dynamic or nonlinear systems. The suggested MDEOA addresses these issues by introducing improved crossover, mutation, and adaptive control techniques that increase population diversity and hasten convergence toward ideal PID gains. To give a methodical comparison analysis, the PID tuning problem is subjected to both the traditional CDE and the suggested MDEOA. The MDEOA-based PID controller offers noticeably better dynamic performance, as shown by analytical simulations and experimental validation. In particular, it completely eliminates the maximum peak and steady-state error, corresponding to 100% reduction in both parameters. The MDEOA-based PID controller reduces the rise time by 10.18%, the peak time by 49.70%, and the settling time by 93.47% compared with the conventional PID controller. These enhancements are a direct result of the algorithm's modified operators' efficacy. Overall, the findings show that MDEOA is a strong and dependable substitute for conventional CDE in intelligent control system parameter optimization. MDEOA is a viable tool for future applications in sophisticated automation and real-time control contexts because of the large improvements in response characteristics, which demonstrate that the suggested alterations greatly increase the resilience, accuracy, and flexibility of PID tuning.

Keywords: Evolutionary algorithms; Classical differential evolution; Modified Differential Evolution Optimization Algorithm; Linear controller optimization

1 Introduction

Classical Differential Evolution (CDE), created by Rainer Storn and Kenneth Price in the mid-1990s, is a type of algorithm that works with groups of possible solutions to find the best answer for problems that involve continuous values. It is known for being easy to use and very good at solving difficult and complex optimization problems that have a lot of non-linear parts [1]. The CDE algorithm generates new solution options by using scaled differences between randomly selected individuals in the population, which mimics the idea of "survival of the fittest" from evolution. This method of optimization uses randomness and is based on how populations interact. It combines both competition and teamwork among members to guide the search for better solutions. It uses real numbers for encoding, has a mutation method that changes differences, follows a survival rule where each individual is replaced by one new one, and has a memory limit, which together make the computational work of standard genetic methods easier. CDE is known for its robust global exploration ability, flexibility in changing environments, and skill in making real-time search modifications. Its key advantages consist of minimal tunable parameters, effective evasion of local minima, and

quick convergence rate [2–4]. CDE shows remarkable efficiency in addressing non-differentiable, nonlinear, high-dimensional, and multimodal optimization challenges. Its algorithmic structure allows for parallel computation, making it very appropriate for issues with computationally intensive objective functions. The CDE process is straightforward and includes four essential steps: The algorithm involves four key phases: initializing the population, mutation, crossover, and selection. Initially, a random set of candidate solutions is generated. The mutation process creates new solution vectors by incorporating a scaled difference from two different, randomly chosen vectors into a selected base vector. In the crossover phase, this altered vector is combined with the original vector to form a trial solution. During the selection stage, the trial vector's performance is assessed in comparison to its parent, and only the one with superior performance progresses to the next generation. This repetitive process persists until a specified stopping criterion is achieved. Despite CDE's proven simplicity and effectiveness, its performance continues to rely heavily on the adjustment of control parameters like the Mutation Factor (MF) and crossover rate. Incorrect parameter settings can lead to sluggish convergence or getting stuck in local minima. To address these challenges, numerous adaptations of the traditional CDE have been created to enhance its stability and effectiveness. These progressions center on the dynamic adjustment of control parameters, improving population diversity, and incorporating hybrid strategies to maintain a balance between exploration and exploitation [5, 6]. Table 1 summarizes representative studies on CDE modifications and the corresponding strategy approaches. Motivated by these factors, this research proposes a Modified Differential Evolution Optimization Algorithm (MDEOA) that incorporates novel modifications to the mutation, crossover, and parameter adaptation mechanisms. The main goal is to enhance convergence speed and optimization accuracy, especially regarding the adjustment of proportional–integral–derivative (PID) controller parameters in optimizing control systems.

Table 1. Overview of research on Classical Differential Evolution (CDE) modifications and strategy approaches

No.	Name	Subject	Problem
1	[7]	Electromagnetism Mutation Factor (MF)	Adjusts MF
2	[8]	Trigonometric mutation	Probabilistic step
3	[9]	CDE/rand/1/either-or	Generates mutant vectors
4	[10]	CDE/current-to-gr.best/1	Vector with crossover
5	[11]	Hybrid algorithm	Description of hybrid CDE with estimation-of-distribution strategy
6	[12]	Neighbourhood-based mutation	Combines strategies
7	[13]	CDE modification	Self-adaptation + mutation combination
8	[14]	Faster-Convergence Mutation Strategy	Adaptive Greediness Control
9	[15]	New CDE/current/1 strategy	Combines strategies + randomization

2 Optimization Strategy in Classical Differential Evolution

In CDE, the optimization method focuses on locating the maximum or minimum of a designated $f(x)$ as an objective function. This challenge presents considerable complexity for various reasons: the vast scope of the search space renders complete examination computationally unfeasible; the objective function might exhibit fluctuating or noisy traits, leading to several possible optima; and set constraints may additionally restrict the viability of specific candidate solutions. CDE addresses these challenges with a strong and adaptable framework tailored for ongoing numerical optimization tasks. Originally suggested by Kenneth in 1994, CDE has gradually evolved into a reliable global optimization technique that can identify genuine optimal solutions with fairly low computational expense. The optimization process in CDE can be mathematically represented in the following way, with the goal of finding the optimal solution X^* in an N -dimensional search space.

$$X^* = [x_1^*, x_2^*, x_N^*] \in D_N = D_1 \cap D_2 \cap \dots \cap D_N \quad (1)$$

This optimal solution must satisfy the conditions.

$$f_{i \min}(X^*) \leq f_{i \min}(X), \forall x = [x_1, x_2, \dots, x_N] \in D_N, 1 \leq i \leq N f_{\min} \quad (2)$$

$$f_{i \max}(X^*) \geq f_{i \max}(X), \forall x = [x_1, x_2, \dots, x_N] \in D_N, 1 \leq i \leq N f_{\max} \quad (3)$$

where, X^* is the best solution within domain D , N is the set of parameters involved, and $f_{i \min}$, $f_{i \max}$ represent the bounds of the objective function for the i -th dimension. DE operates on a population matrix $P_{ij} \in R[D \times N_P]$, where each solution vector is initialized under predefined boundary constraints $L \leq P_{ij} \leq H$, using:

$$P_{ij} = L + (H - L) \cdot \text{rand}_{ij}[0, 1] \quad (4)$$

To initialize individuals $x_{ij}(0)$ DE applies:

$$x_{ij}(0) = L_{ij} + (U_{ij} - L_{ij}) \cdot \text{rand}_{ij}(0, 1) \quad (5)$$

This ensures a diverse and unbiased initial population. In the mutation step, three distinct vectors V_1, V_2, V_3 are selected randomly from the current population such that $V_1 \neq V_2 \neq V_3$, and the mutation vector V_y is computed as:

$$V_y = V_3 + F \cdot (V_1 - V_2) \quad (6)$$

where, F represents the scaling factor generally within the range $[0, 2]$. CDE includes multiple mutation strategies designed for diverse optimization problems [16]. The most prevalent consist of:

CDE/rand/1:

$$x_{r1}(t) + F(x_{r2}(t) - x_{r3}(t)) = h_i(t + 1) \quad (7)$$

CDE/best/1:

$$x_{\text{best}}(t) + F(x_{r1}(t) - x_{r2}(t)) = h_i(t + 1) \quad (8)$$

CDE/target-to-best/1:

$$x_i(t) + F(x_{\text{best}}(t) - x_i(t)) + F(x_{r1}(t) - x_{r2}(t)) = h_i(t + 1) \quad (9)$$

CDE/rand/2:

$$x_{r1}(t) + F(x_{r2}(t) - x_{r3}(t)) + F(x_{r4}(t) - x_{r5}(t)) = h_i(t + 1) \quad (10)$$

CDE/best/2:

$$x_{\text{best}}(t) + F(x_{r1}(t) - x_{r2}(t)) + F(x_{r4}(t) - x_{r5}(t)) = h_i(t + 1) \quad (11)$$

These approaches create new potential solutions by merging chosen base vectors with adjusted difference vectors, thus enhancing the likelihood of reaching the global optimum. The notation typically utilized to depict the CDE mutation scheme is stated as follows:

$$\text{CDE}/x/y/z \quad (12)$$

In this notation, x denotes the base vector approach (like rand, best, or current), y signifies the number of difference vectors, and z indicates the type of crossover (binomial or exponential) [12, 17–19]. Although many CDE strategies utilize intricate hybrid or adaptive methods that depend significantly on stochastic components, this study presents a straightforward mutation technique that takes advantage of beneficial historical trends and foundational vectors. In contrast to traditional approaches, the suggested method steers clear of random choice of vectors and scaling factors, focusing instead on prior successful search paths. This results in a more predictable and computationally efficient method, with initial tests validating improved convergence and better solution quality.

3 Optimization Strategy in Modified Differential Evolution Optimization Algorithm

The MDEOA expands the traditional CDE method with improvements designed to enhance effectiveness in intricate optimization challenges, particularly in control-focused scenarios like intelligent PID controller adjustment. Key advancements feature a dynamic mutation approach that modifies based on previous success rates, self-evolving control parameters that see both the MF and crossover rate develop across iterations, and an adaptive mutation rate that improves the equilibrium between exploration and exploitation across the search space [20].

4 Algorithm Operations in Modified Differential Evolution Optimization Algorithm

4.1 Crossover Scheme Modification

Ensures diversity by determining whether the mutant vector component $v_{ij}(t + 1)$ is selected over the current individual:

If

$$\text{rand}_{ij}(0, 1) \leq C_R, h_{ij}(t + 1) = v_{ij}(t + 1) \quad (13)$$

Else

$$h_{ij}(t + 1) = x_{ij}(t) \quad (14)$$

The crossover-rate parameter η is dynamically adjusted over generations using a sigmoid function:

$$C_R = C_r = \eta = \eta_{\min} + \left[(\eta_{\max} - \eta_{\min}) / \left(1 + e^{-\alpha^*(K - K_{\max})} \right) \right] \quad (15)$$

where, $\eta_{\min}$ and $\eta_{\max}$ are the minimum and maximum crossover-rate parameter probabilities. α is the sigmoid slope factor. K is the present current iteration index. $K_{\max}$ is the maximum number of iterations.

4.2 Selection Scheme Modification

The selection operation chooses the better individual between the trial vector $h_i(t+1)$ and the current individual $x_i(t)$ based on the fitness function $f(\cdot)$:

$$x_i(t+1) = h_i(t+1), f(h_i(t+1)) < f(x_i(t)) \quad (16)$$

$$x_i(t+1) = x_i(t), f(h_i(t+1)) \geq f(x_i(t)) \quad (17)$$

The utilization of MDEOA for adjusting PID controllers in control systems involves several essential steps. The problem is first outlined by creating objective functions that focus on minimizing vibrations and enhancing energy efficiency, while also detailing constraints and the search space. The optimization framework is formulated by defining objective functions, decision variables, constraints, and search-space boundaries appropriate for the optimization problem under consideration. The PID controller starts with initial gain values, and then MDEOA progressively fine-tunes these parameters using its enhanced mutation and crossover techniques. Ultimately, the control signals are refreshed with the optimized gains, and the vibration suppression and energy efficiency of the system are evaluated.

4.3 Mutation Scheme Modification

Two strategies of classical mutation are blended:

CDE/rand/1 (diversity-focused):

$$\text{CDE/rand/1} = M_1(\text{time} + 1) = [N(K)^* [x_2(\text{time}) - x_3(\text{time})]] + [x_1(\text{time})] \quad (18)$$

CDE/best/1 (convergence-focused):

$$\text{CDE/best/1} = M_2(\text{time} + 1) = [N(K)^* [x_1(\text{time}) - x_2(\text{time})]] + [x_{\text{best}}(\text{time})] \quad (19)$$

The final hybrid mutation vector is:

$$M_{\text{New}}(\text{time} + 1) = [(1 - \zeta)]^* [[M_2(\text{time} + 1)] + [\epsilon^* \text{randn}]] + [\zeta]^* [[M_1(\text{time} + 1)] + [\epsilon^* \text{randn}]] \quad (20)$$

where, ζ is the weighting factor controlling the contribution of the CDE/rand/1 mutation strategy, and ϵ is the Gaussian noise factor. $N(K)$ is the adaptive population-size scaling function used to adjust the population size during the optimization process. The concept of combining multiple mutation strategies to improve search diversity and convergence behavior has previously been explored in differential evolution research. Hybrid mutation mechanisms have demonstrated superior performance compared with single-strategy approaches by balancing exploration and exploitation throughout the optimization process [7], $CR/Cr/\eta$ are as crossover-rate parameter.

4.4 Variance Factor Scheme Modification

The Variation Factor (VF), referred to as F, is an essential element that influences the search process in the CDE, algorithm, impacting population diversity and convergence traits. As the MF increases, the mutation step size becomes larger, which generally increases population diversity and enhances exploration, although excessive values may reduce convergence stability. Conversely, raising the MF leads to greater diversity in the population, thereby enhancing the chance for the algorithm to “escape” from extreme values. Since the Variation Factor (VF), F, is typically held constant within the interval [0, 2], the conventional differential evolution method does not effectively leverage the evolutionary dynamics exhibited at various stages of the algorithm’s development. Consequently, in this study, the number of iterations is crucial in establishing the Variation Factor, as shown in Eq. (21) and Eq. (22):

$$N(K) = e^{-[[1-K]/[K m-1]]^* [1+[K/K_m]]} \quad (21)$$

$$N(K) = \left[\left[1 / \left(1 + \left[\beta * e^{-\zeta_1 * [K - K_{\text{max}}/2]} \right] \right) * [2N_{\text{ave}}] \right] + N_{\text{min}} \right] * [R(K)] \quad (22)$$

where, ζ_1 is the damping parameter controlling the rate of adjustment of $N(K)$. In this framework, the key parameters include ζ_1 (Damping parameter), N_{max} (maximum population size), N_{min} (minimum population size), and N_{ave} (average/reference population size). The constants for gain or gravity are denoted by K as current iteration index, which can be referred to as K_{max} or K_m in terms of overall gain. Stochastic variations are included via ϵ -randn. Variables associated with the iterative process include K representing the current iteration index, K_{max} denoting the maximum population size allowed, and K_m as specific iteration/control parameter. Moreover, the coefficients

α , β , and ϵ are essential in managing and enhancing the efficiency of the *DE* algorithm. In particular, α regulates the crossover likelihood, where increased values may lead to more sudden shifts. In the same way, β affects the variance factor, with higher values leading to a more gradual adjustment curve. The coefficient ϵ introduces Gaussian noise, adding randomness that enhances exploration. The recommended ranges for these coefficients are α from 0.5 to 2.0, β from 0.1 to 0.5, and ϵ from 0.1 to 1.0. While executing, every parameter is randomly modified within its defined limits, with `rand()` generating random values scaled based on the problem's features and the necessary balance between exploration and exploitation.

5 Methodology in Modified Differential Evolution Optimization Algorithm

Crossover is utilized on the existing population, substituting members when the new candidates show improved performance. In the proposed MDEOA, the *DE* operations are performed in the following sequence: initialization, mutation, crossover, and selection. First, the initial population is generated to provide adequate coverage of the search space. The mutation operation generates mutant vectors based on the differences among individuals in the population. Subsequently, crossover combines the mutant vectors with the current individuals according to the crossover rate to generate trial vectors. Finally, the selection operation compares the trial vector with the current individual based on the fitness value, and the better solution is retained for the next generation. The performance is evaluated using the Integral Absolute Error (IAE) and the optimized PID parameters. To enhance performance, a distributed exponential function utilizing the subsequent probability density function:

$$f(x; \lambda) = \lambda e^{-(\lambda x)}, x \geq 0 \quad (23)$$

$$f(x; \lambda) = 0, x < 0 \quad (24)$$

The distribution is characterized by λ being the reciprocal of the average. By directing the population elements towards the bottom area of this distribution, the total error is diminished, which subsequently improves the algorithm's performance efficiency. The enhanced CDE, algorithm includes significant improvements that provide distinct benefits compared to the traditional CDE approach, particularly in terms of dynamic adaptability, diversity preservation, and convergence performance. These enhancements together lead to a stronger and smarter optimization performance. The algorithm specifically substitutes the fixed mutation operators found in traditional CDE, like CDE /rand/1 and CDE /best/1, with an innovative hybrid mutation approach. This method harmonizes exploration and exploitation with a weighted mix of M_1 and M_2 , controlled by an adaptive parameter ζ , while adding stochastic disturbance through Gaussian noise (ϵ -randn). This method improves population diversity and reduces the chances of early convergence, a common drawback of traditional CDE. Moreover, the mutation scaling is dynamically adjusted by the adaptive function $R(K)$, which varies with current iteration index K . This presents a nonlinear exponential decay that adjusts the impact of mutations over time, preserving significant diversity in initial iterations and allowing for accurate convergence in subsequent phases. In contrast, conventional CDE typically uses fixed or linearly diminishing scaling factors. Moreover, the population size is adjusted in real-time using the function $N(K)$, setting this technique apart from traditional CDE. Through the integration of adaptive control utilizing parameters like β , ζ , $K_{\max}$, and N_{ave} , the algorithm adjusts the count of candidate solutions as evolution unfolds. This self-adjusting population system reconciles computational efficiency with solution variety, in contrast to the fixed population size in conventional CDE. Moreover, employing a sigmoid-like mutation scaling factor η to control mutation intensity adds another dimension of sophistication, guaranteeing a seamless shift between exploration and exploitation stages and allowing the algorithm to focus on high-potential areas of the solution space as convergence advances. The advantages of this altered CDE method compared to traditional CDE are clear in multiple areas: it enhances the exploration-exploitation balance through adaptive hybrid mutation and dynamic scaling functions, speeds up convergence by prioritizing promising solutions while steering clear of local minima, and implements dynamic population control to optimize search pressure and computational resource distribution during the optimization process. The threat of early convergence is reduced by maintaining diversity within the population via stochastic elements and adaptive weighting. In general, the suggested algorithm exhibits improved robustness and adaptability, rendering it more efficient for various intricate optimization issues compared to traditional CDE [15, 21–23], as shown in Figures 1–4. Initially, the population is formed using a uniform distribution, ensuring a consistent scattering of individuals throughout the search space. As the CDE algorithm evolves, these individuals gradually come together, eventually resulting in the discovery of the best solution.

Utilizing a larger population size enhances the chances of achieving the optimal solution; thus, as illustrated in Figure 1, the initial population for this research has been created. Figure 2 illustrates the application of a distributed exponential function to achieve optimal performance.

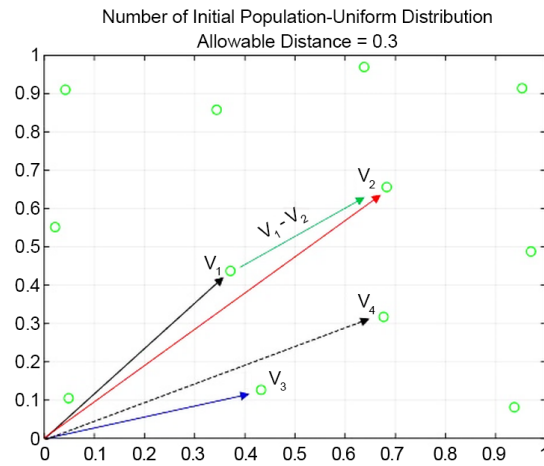


Figure 1. Initial population with 0.3 distance in Modified Differential Evolution Optimization Algorithm (MDEOA)

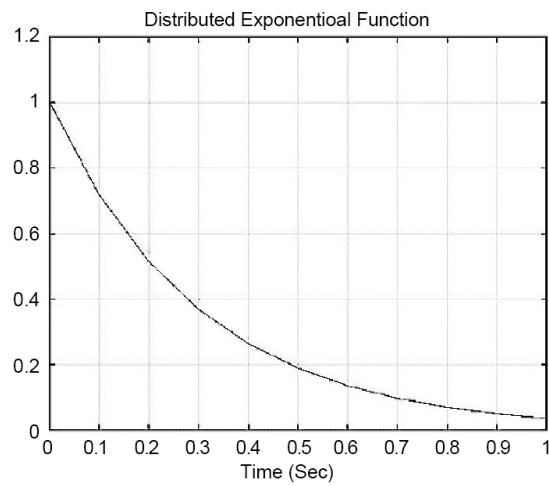


Figure 2. Exponential function distribution in Modified Differential Evolution Optimization Algorithm (MDEOA)

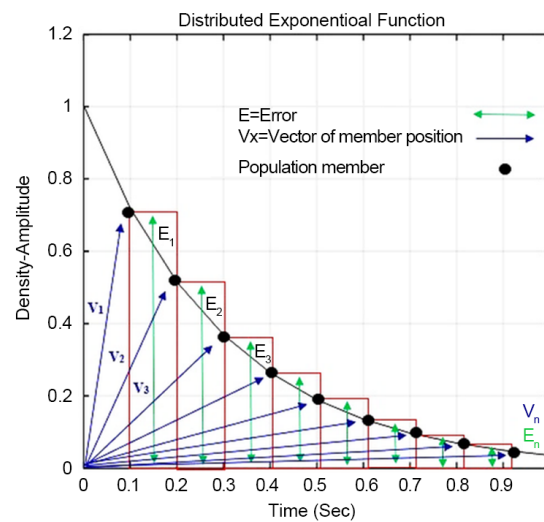


Figure 3. Exponential function distribution and mutation strategy in Modified Differential Evolution Optimization Algorithm (MDEOA)

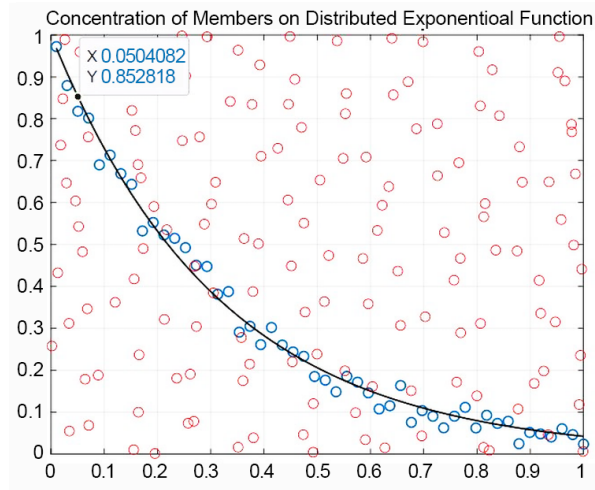


Figure 4. Member concentration on exponential distribution in Modified Differential Evolution Optimization Algorithm (MDEOA)

This ongoing, purely decreasing distribution allows the objective function to reduce error rates while enhancing algorithm performance, as illustrated in Figure 3. When individuals in the population are arranged based on this exponential distribution, showing a greater density at the lower end, the error rate is reduced in the CDE strategy and algorithm suggested in this study. This configuration brings the PID control parameters closer to their ideal values, thus improving the overall performance of the system. The effectiveness of this approach is confirmed in Figure 4 [24–30].

Figures 1–4 demonstrate the progressive evolution of the population distribution throughout the optimization process. The concentration of individuals near the lower region of the exponential distribution indicates increased exploitation of promising search regions while preserving adequate diversity during early iterations. This behavior contributes directly to faster convergence and improved solution quality achieved by the proposed MDEOA.

6 Evaluation of the Modified Differential Evolution Optimization Algorithm Compared with Classical Differential Evolution

As shown in Figures 5–7, the modified DE algorithm (MDEOA) was evaluated in MATLAB through two different methods for this research.

The effectiveness of the proposed method was first assessed on three mathematical benchmark problems from CEC'14. All experiments were conducted over 30 independent runs, with a maximum of 200 iterations per run. The results reported in Tables 2–4 summarize the comparative performance of CDE and MDEOA in terms of iteration and fitness values. The MATLAB code was altered for both the MDEOA and traditional DE algorithms, ensuring that both utilized the same initialized population. MDEOA achieves a 50% improvement over CDE in terms of the minimum number of iterations.

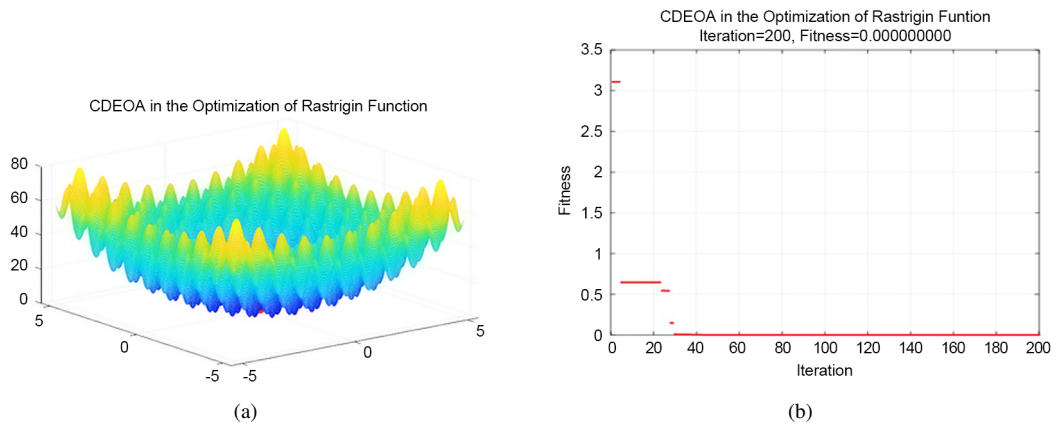


Figure 5. Results 1 and 2 of Classical Differential Evolution (CDE) optimization of the rastrigin function with parameters in Table 2: (a) result 1; (b) result 2

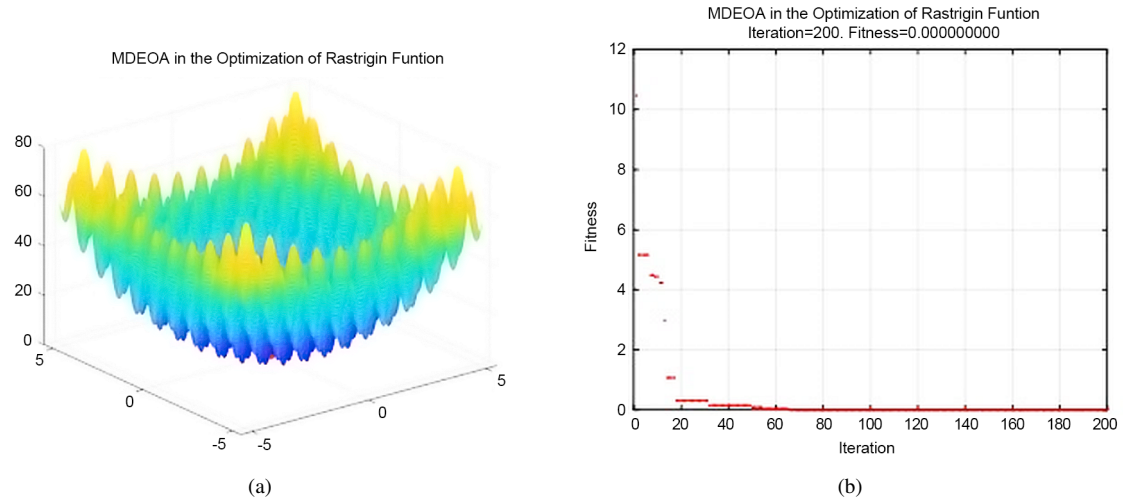


Figure 6. Results 1 and 2 of Modified Differential Evolution Optimization Algorithm (MDEOA) optimization of rastrigin function with parameters Table 2: (a) result 1; (b) result 2

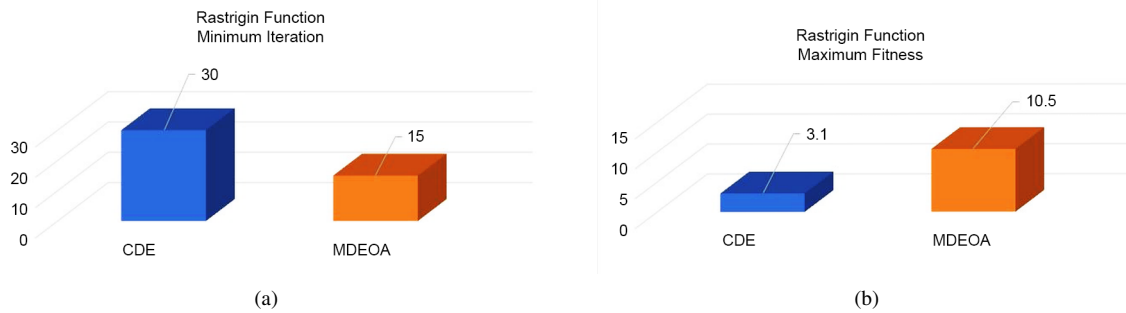


Figure 7. Minimum iteration and maximum fitness of comparison of Classical Differential Evolution (CDE) and Modified Differential Evolution Optimization Algorithm (MDEOA) in optimizing the rastrigin function: (a) minimum iteration; (b) maximum fitness

Table 2. Comparative results of mathematical problems obtained using Classical Differential Evolution (CDE) and Modified Differential Evolution Optimization Algorithm (MDEOA) [20]

Fixed Parameters Set	CDE	MDEOA
$F = 0.5, C = 0.9, D = 2, N_P = 50$	Rastrigin function	Rastrigin function
Maximum iteration	200	200
Minimum iteration	30	15
Maximum fitness	3.1	10.5
Minimum fitness	0	0

Table 3. Percentage improvement of Modified Differential Evolution Optimization Algorithm (MDEOA) compared to Classical Differential Evolution (CDE) at the minimum number of iterations [20]

Function	Minimum Iteration Improvement in MDEOA
Rastrigin	50%
Sphere	50%
Sum square's	50%

Table 4. Percentage improvement of Classical Differential Evolution (CDE) compared to Modified Differential Evolution Optimization Algorithm (MDEOA) in terms of maximum fitness [20]

Function	Maximum Fitness Improvement in CDE
Rastrigin	70.50%
Sphere	57.50%
Sum square's	83%

The results were later charted in MATLAB, showing the best fitness value, the associated solution position, and the progression over iterations. Each benchmark function was tested with both algorithms using identical parameter settings to maintain a fair comparison. The enhanced CDE algorithm was assessed using PlatEMO, a MATLAB-based environment for evolutionary computation developed by Cuate for the evaluation of optimization algorithms.

PlatEMO is a well-known and functional toolkit that allows researchers to choose from 176 diverse algorithms, test 345 benchmark problems, modify algorithm parameters, and produce statistical summaries along with visual performance graphs. Tian explains that it offers an extensive framework for evaluating algorithm performance across a variety of established mathematical test functions.

Figure 7 highlights the superior convergence characteristics of the proposed MDEOA. The reduction in the required number of iterations indicates faster optimization, while the lower fitness values demonstrate improved optimization performance for the minimization benchmark functions. These results suggest that the adaptive mutation and crossover mechanisms effectively balance exploration and exploitation throughout the evolutionary process.

7 Proportional-Integral-Derivative Evaluation

As PID Controller Performance Evaluation, To evaluate the effectiveness of the proposed MDEOA in PID parameter tuning, standard control performance indices were considered, including rise time (T_r), settling time (T_s), maximum overshoot (M_p), and steady-state error (E_{ss}). These performance measures are commonly employed to assess controller quality in dynamic systems. The optimization objective is to determine the proportional gain (K_p), integral gain (K_i), and derivative gain (K_e) that minimize system error while maintaining fast response and stability. The IAE was utilized as the principal performance criterion during optimization. A lower rise time and settling time indicate faster system response, whereas reduced overshoot and steady-state error reflect improved stability and tracking performance. The proposed MDEOA is specifically designed to improve these control indices through adaptive mutation and crossover mechanisms (Table 5 and Figure 8) [24–30]. For comparison with the proposed MDEOA-based PID tuning approach, the previously reported Active Vibration Control (AVC)-CDE-PID results from our earlier study [20] are used as reference results.

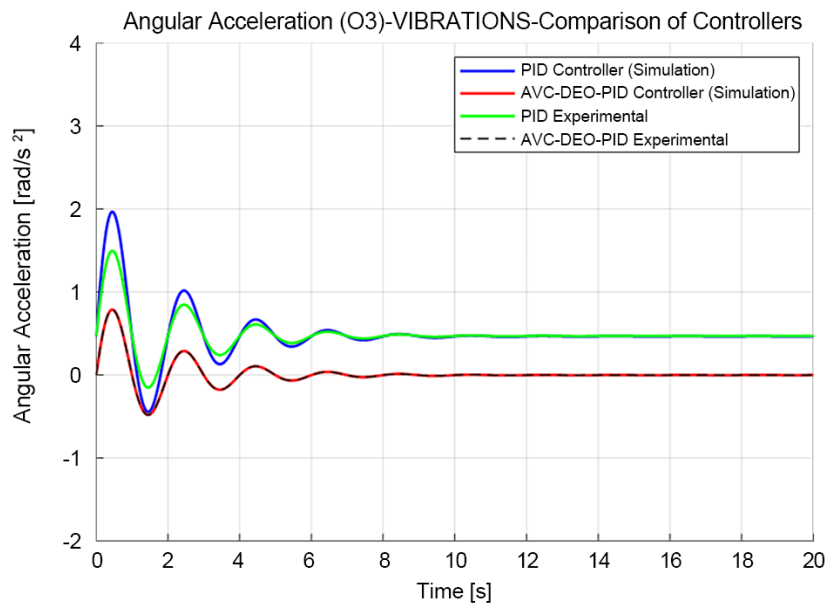


Figure 8. Angular acceleration (vibrations) comparison of controllers in Mechatronics Test Rig System (MTRS)

Note: PID = proportional-integral-derivative; AVC = Active Vibration Control.

Table 5. Previously reported simulation results for PID and AVC-CDE-PID controllers from our earlier study [20]

MTRS Parameters Set	PID	AVC-CDE-PID
T_R	1.002	0.9
T_P	2.008	1.01
T_S	15.332	1.001
M_P	0.294	0.00
E_{SS}	0.467	0.000

Note: MTRS = Mechatronics Test Rig System; AVC = Active Vibration Control; CDE = Classical Differential Evolution; PID = proportional-integral-derivative.

Figure 8 presents the previously reported AVC-CDE-PID as AVC-DEO-PID results from our earlier study [20], which are used here for comparison. In addition to CDE, advanced variants such as Adaptive Differential Evolution with Optional External Archive (JADE) and Success-History based Adaptive Differential Evolution (SHADE) improve adaptive parameter control and convergence. JADE uses adaptive mutation with external archives, while SHADE applies historical memory for parameter updates. The proposed MDEOA introduces adaptive crossover scaling, hybrid mutation, and dynamic population regulation to enhance convergence and maintain diversity. Future work will compare MDEOA with JADE, SHADE, and other CDE variants under identical benchmark conditions.

8 Conclusion

This study presents a MDEOA that greatly enhances the adjustment of PID controller parameters compared to the CDE method. Through the enhancement of mutation, crossover, and control strategies, the MDEOA addresses critical limitations of CDE, including premature convergence, rigid mutation patterns, and a constant population size, utilizing dynamic and adaptive mechanisms. Main innovations consist of a hybrid mutation approach, nonlinear exponential decay for scaling purposes, self-regulating population management, and stochastic diversity augmentation, which collectively guarantee a strong equilibrium between exploration and exploitation throughout the optimization process. Simulation and experimental findings indicate that MDEOA enhances convergence rate and solution precision while markedly boosting control performance metrics such as T_r , T_s , and e_{ss} . Comparative evaluations in MATLAB utilizing CEC'14 benchmark functions and platforms such as PlatEMO demonstrate that MDEOA consistently exceeds classical DE regarding robustness and adaptability. The results demonstrate that the MDEOA-based PID controller improves the dynamic response, achieving reductions of 10.18% in rise time, 49.70% in peak time, and 93.47% in settling time. In addition, the maximum peak and steady-state error are completely eliminated, corresponding to 100% reduction in both parameters. These results emphasize the considerable promise of MDEOA as an important breakthrough in control engineering and evolutionary algorithms. The benchmark results demonstrate the effectiveness of the proposed MDEOA in solving nonlinear optimization problems. The obtained performance improvements indicate strong potential for future application in PID controller tuning and advanced control system optimization.

Author Contributions

Development, implementation, and validation of the proposed intelligent metaheuristic algorithm for the optimal tuning of control system parameters, A.M., A.A., T.S., and R.K.; research leadership, algorithm development, computational and experimental work, data analysis, and manuscript preparation, A.M.; technical supervision, validation, and critical review, A.A., T.S., and R.K. All authors have read and agreed to the published version of the manuscript.

Data Availability

The data used to support the findings of this study are available from the corresponding author upon request.

Conflicts of Interest

The authors declare no conflicts of interest.

References

- [1] J. S. Vesterström and R. Thomsen, "A comparative study of differential evolution, particle swarm optimization, and evolutionary algorithms on numerical benchmark problems," in *Proceedings of the 2004 Congress on Evolutionary Computation*, Portland, OR, USA, 2004, pp. 1980–1987. <https://doi.org/10.1109/CEC.2004.1331139>

- [2] R. Storn and K. Price, "Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces," *J. Glob. Optim.*, vol. 11, no. 4, pp. 341–359, 1997. <https://doi.org/10.1023/A:1008202821328>
- [3] S. Das and P. N. Suganthan, "Differential evolution: A survey of the state-of-the-art," *IEEE Trans. Evol. Comput.*, vol. 15, no. 1, pp. 4–31, 2010. <https://doi.org/10.1109/TEVC.2010.2059031>
- [4] J. Brest, S. Greiner, B. Bošković, M. Mernik, and V. Žumer, "Self-adapting control parameters in differential evolution: A comparative study on numerical benchmark problems," *IEEE Trans. Evol. Comput.*, vol. 10, no. 6, pp. 646–657, 2006. <https://doi.org/10.1109/TEVC.2006.872133>
- [5] V. Feoktistov, *Differential Evolution: In Search of Solutions*. Boston, MA: Springer, 2006. https://doi.org/10.1007/978-0-387-36896-2_1
- [6] A. W. Mohamed and A. S. Almazayad, "Differential evolution with novel mutation and adaptive crossover strategies for solving large scale global optimization problems," *Appl. Comput. Intell. Soft Comput.*, vol. 2017, no. 1, p. 7974218, 2017. <https://doi.org/10.1155/2017/7974218>
- [7] P. Kaelo and M. M. Ali, "Differential evolution algorithms using hybrid mutation," *Comput. Optim. Appl.*, vol. 37, no. 2, pp. 231–246, 2007. <https://doi.org/10.1007/s10589-007-9014-3>
- [8] H. Y. Fan and J. Lampinen, "A trigonometric mutation operation to differential evolution," *J. Glob. Optim.*, vol. 27, no. 1, pp. 105–129, 2003. <https://doi.org/10.1023/A:1024653025686>
- [9] F. Neri and V. Tirronen, "Recent advances in differential evolution: A survey and experimental analysis," *Artif. Intell. Rev.*, vol. 33, no. 1, pp. 61–106, 2010. <https://doi.org/10.1007/s10462-009-9137-2>
- [10] Q. Fan and X. Yan, "Self-adaptive differential evolution algorithm with zoning evolution of control parameters and adaptive mutation strategies," *IEEE Trans. Cybern.*, vol. 46, no. 1, pp. 219–232, 2015. <https://doi.org/10.1109/TCYB.2015.2399478>
- [11] B. H. Zhou, L. M. Hu, and Z. Y. Zhong, "A hybrid differential evolution algorithm with estimation of distribution algorithm for reentrant hybrid flow shop scheduling problem," *Neural Comput. Appl.*, vol. 30, no. 1, pp. 193–209, 2018. <https://doi.org/10.1007/s00521-016-2692-y>
- [12] M. F. Ahmad, N. A. M. Isa, W. H. Lim, and K. M. Ang, "Differential evolution: A recent review based on state-of-the-art works," *Alex. Eng. J.*, vol. 61, no. 5, pp. 3831–3872, 2022. <https://doi.org/10.1016/j.aej.2021.09.013>
- [13] K. Gokul, R. Pooja, K. Gowtham, and G. Jeyakumar, "A self-switching base vector selection mechanism for differential mutation of differential evolution algorithm," in *2017 International Conference on Communication and Signal Processing (ICCSP)*, Chennai, India, 2017, pp. 1545–1549. <https://doi.org/10.1109/ICCSP.2017.8286647>
- [14] W. J. Yu, J. J. Li, J. Zhang, and M. Wan, "Differential evolution using mutation strategy with adaptive greediness degree control," in *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, 2014, pp. 73–80. <https://doi.org/10.1145/2576768.2598236>
- [15] W. L. Xiang, X. L. Meng, M. Q. An, Y. Z. Li, and M. X. Gao, "An enhanced differential evolution algorithm based on multiple mutation strategies," *Comput. Intell. Neurosci.*, vol. 2015, no. 1, p. 285730, 2015. <https://doi.org/10.1155/2015/285730>
- [16] A. K. Qin, V. L. Huang, and P. N. Suganthan, "Differential evolution algorithm with strategy adaptation for global numerical optimization," *IEEE Trans. Evol. Comput.*, vol. 13, no. 2, pp. 398–417, 2008. <https://doi.org/10.1109/TEVC.2008.927706>
- [17] T. Eltaieb and A. Mahmood, "Differential evolution: A survey and analysis," *Appl. Sci.*, vol. 8, no. 10, p. 1945, 2018. <https://doi.org/10.3390/app8101945>
- [18] D. Cruze, G. Hemalatha, S. V. S. Jebadurai, L. Sarala, D. Tensing, and S. J. E. Christy, "A review on the magnetorheological fluid, damper and its applications for seismic mitigation," *Civ. Eng. J.*, vol. 4, no. 12, pp. 3058–3074, 2018. <https://doi.org/10.28991/cej-03091220>
- [19] M. Leon and N. Xiong, "Investigation of mutation strategies in differential evolution for solving global optimization problems," in *International Conference on Artificial Intelligence and Soft Computing*, 2014, pp. 372–383. https://doi.org/10.1007/978-3-319-07173-2_32
- [20] A. Moloody, A. As'array, T. S. Hong, R. Kamil, M. Z. M. Zain, and I. Bahiuddin, "An experimental study on the mechatronics design and implementation of an intelligent active vibration control system for a CSLRFM," *J. Adv. Res. Appl. Mech.*, vol. 134, no. 1, pp. 61–89, 2025. <https://doi.org/10.37934/aram.134.1.6189>
- [21] J. Zhang and A. C. Sanderson, "JADE: Adaptive differential evolution with optional external archive," *IEEE Trans. Evol. Comput.*, vol. 13, no. 5, pp. 945–958, 2009. <https://doi.org/10.1109/TEVC.2009.2014613>
- [22] M. Abbas, A. Azizan, S. H. Tang, K. Raja, and Y. Zuhri, "A comprehensive review based on analysis, modelling, mechanical vibration control strategies, and optimization methods of robotics flexible manipulator structures," *Pertanika J. Sci. Technol.*, vol. 33, no. 1, pp. 21–45, 2025. <https://doi.org/10.47836/pjst.33.1.02>

- [23] R. Tanabe and A. Fukunaga, "Success-history based parameter adaptation for differential evolution," in *2013 IEEE Congress on Evolutionary Computation*, Cancun, Mexico, 2013, pp. 71–78. <https://doi.org/10.1109/CEC.2013.6557555>
- [24] M. Fan, J. Chen, Z. Xie, H. Ouyang, S. Li, and L. Gao, "Improved multi-objective differential evolution algorithm based on a decomposition strategy for multi-objective optimization problems," *Sci. Rep.*, vol. 12, no. 1, p. 21176, 2022. <https://doi.org/10.1038/s41598-022-25440-7>
- [25] Q. Yang, S. C. Chu, J. S. Pan, J. H. Chou, and J. Watada, "Dynamic multi-strategy integrated differential evolution algorithm based on reinforcement learning for optimization problems," *Complex Intell. Syst.*, vol. 10, no. 2, pp. 1845–1877, 2024. <https://doi.org/10.1007/s40747-023-01243-9>
- [26] Z. Zhang, J. Zhu, and F. Nie, "A novel hybrid adaptive differential evolution for global optimization," *Sci. Rep.*, vol. 14, no. 1, p. 19697, 2024. <https://doi.org/10.1038/s41598-024-70731-w>
- [27] S. M. Azzam, O. E. Emam, and A. S. Abolaban, "An improved differential evolution with sailfish optimizer (DESFO) for handling feature selection problem," *Sci. Rep.*, vol. 14, no. 1, p. 13517, 2024. <https://doi.org/10.1038/s41598-024-63328-w>
- [28] B. Subburaj, J. U. Maheswari, S. S. Ibrahim, and M. S. Kavitha, "Population diversity control based differential evolution algorithm using fuzzy system for noisy multi-objective optimization problems," *Sci. Rep.*, vol. 14, no. 1, p. 17863, 2024. <https://doi.org/10.1038/s41598-024-68436-1>
- [29] N. Liu, J. Luo, J. Chang, and J. S. Pan, "Differential evolution using multi-strategy for the improvement of optimization performance," *Neural Comput. Appl.*, vol. 37, no. 27, pp. 22 593–22 620, 2025. <https://doi.org/10.1007/s00521-024-10781-3>
- [30] R. Zhong, E. Zhang, and M. Munetomo, "Cooperative coevolutionary surrogate ensemble-assisted differential evolution with efficient dual differential grouping for large-scale expensive optimization problems," *Complex Intell. Syst.*, vol. 10, no. 2, pp. 2129–2149, 2024. <https://doi.org/10.1007/s40747-023-01262-6>